# Serializable Snapshot Isolation (SSI)

## Table of Contents



---

## Learning Objectives

- Understand write skew anomaly and why snapshot isolation doesn't prevent it
- Know how PostgreSQL's SSI uses predicate locks to detect serialization anomalies
- Implement retry logic for serialization failures (SQLSTATE 40001)
- Choose between SERIALIZABLE and REPEATABLE READ based on correctness requirements

---

## Background: Snapshot Isolation and Write Skew

### Snapshot Isolation (REPEATABLE READ)

Under snapshot isolation, each transaction reads from a consistent snapshot taken at transaction start. This prevents dirty reads, non-repeatable reads, and phantom reads — but NOT write skew.

### Write Skew Anomaly

Write skew occurs when two transactions each read overlapping data and write to non-overlapping data, but together violate a constraint that neither transaction individually violated.

**Classic example: On-call doctors**

```
Constraint: at least 1 doctor must be on call at all times.
Currently: Alice on_call=true, Bob on_call=true

T1 (Alice going off call):
  READ: COUNT(*) of on_call doctors = 2  → safe to go off call
  WRITE: UPDATE doctors SET on_call=false WHERE name='Alice'

T2 (Bob going off call, concurrent):
  READ: COUNT(*) of on_call doctors = 2  → safe to go off call
  WRITE: UPDATE doctors SET on_call=false WHERE name='Bob'
```

```
Result: both off call. Constraint violated.
Neither transaction individually violated the constraint at the time of its read.
```

ASCII diagram:

```
T1: [READ count=2 ✓] ──────────────────────── [WRITE Alice=false] [COMMIT]
T2:       [READ count=2 ✓] ──────────────── [WRITE Bob=false]   [COMMIT]
          ↑ both read same snapshot             ↑ write to different rows
```

Under REPEATABLE READ, this write skew is ALLOWED because neither write conflicts with what the other transaction READ (they write different rows).

Under SERIALIZABLE (SSI), this is DETECTED and one transaction is aborted.

---

## How SSI Works

### Predicate Locks (SIReadLock)

SSI tracks not just which rows were written, but which rows (or ranges) were READ. These are called predicate locks or SIRead locks. They don't block — they only track.

When T1 reads rows matching a condition, PostgreSQL records that T1 has a read-dependency on those rows.

### rw-Anti-Dependency Cycle Detection

SSI looks for rw-anti-dependency cycles:

```
An rw-anti-dependency exists when:
  T1 reads a version of X that was written before T2 (T2 wrote X after T1 read it)
  AND
  T2 reads a version of Y that T1 later writes (T1 writes Y after T2 reads it)
  → Cycle: T1 must come before T2 (T1 reads pre-T2 X)
           AND T2 must come before T1 (T2 reads pre-T1 Y)
           → Impossible to serialize → one must abort
```

For the doctor example:

- T1 reads the on_call count (reads rows T2 will write)
- T2 reads the on_call count (reads rows T1 will write)
- Both write different rows
- Creates a cycle: T1 before T2, T2 before T1 → not serializable

### Serialization Failure Error

```
ERROR:  could not serialize access due to read/write dependencies among transactions
DETAIL:  Reason code: Canceled on identification as a pivot, during write.
HINT:   The transaction might succeed if retried.
SQLSTATE: 40001
```

The HINT says it all: **the application must retry on 40001**.

---

## SSI vs REPEATABLE READ

| Anomaly | READ COMMITTED | REPEATABLE READ | SERIALIZABLE (SSI) |
|---|---|---|---|
| Dirty read | ✗ Possible | ✓ Prevented | ✓ Prevented |
| Non-repeatable read | ✗ Possible | ✓ Prevented | ✓ Prevented |
| Phantom read | ✗ Possible | ✓ Prevented* | ✓ Prevented |
| Write skew | ✗ Possible | ✗ Possible | ✓ Prevented |
| Serialization anomaly | ✗ Possible | ✗ Possible | ✓ Prevented |

*PostgreSQL's REPEATABLE READ prevents phantoms due to MVCC snapshot semantics.

### Key Distinction

REPEATABLE READ and SERIALIZABLE both use snapshots. The difference:

- REPEATABLE READ: detects write-write conflicts only
- SERIALIZABLE: also detects read-write anti-dependencies (via predicate locks)

---

## Performance Overhead

SSI adds overhead compared to REPEATABLE READ:

1. **Memory**: predicate locks stored in `pg_locks` (SIReadLock entries). Each read-set range requires memory.
2. **CPU**: cycle detection algorithm runs on commit
3. **Abort rate**: some transactions will be aborted and retried
4. **Safe snapshots**: PostgreSQL tracks the "safe snapshot" — oldest snapshot that can be cleaned up

### Monitoring

```
-- Count serialization failures in current database
SELECT datname, xact_rollback, conflicts
FROM pg_stat_database
WHERE datname = current_database();

-- Active SIRead locks (predicate locks)
SELECT relation::regclass, page, tuple, pid, mode
FROM pg_locks
WHERE locktype = 'relation' AND mode = 'SIReadLock';

-- Transactions currently in serializable mode
SELECT pid, state, isolation_level
FROM pg_stat_activity
WHERE backend_type = 'client backend';

-- Check for serialization failures per session
SELECT pid, query, wait_event_type, wait_event
FROM pg_stat_activity
WHERE state = 'idle in transaction (aborted)';
```

---

## When to Use SERIALIZABLE

Use SERIALIZABLE when **correctness is more important than throughput** and you cannot express the constraint in a single SQL statement.

**Good use cases:**

- Financial double-entry bookkeeping (debit + credit must balance)
- Inventory reservation (prevent overselling)
- Ticket/seat booking (prevent double-booking)
- Any "check then act" pattern involving aggregates
- When application logic assumes transactions run in serial order

**Cases where REPEATABLE READ suffices:**

- Read-only transactions (no writes → no conflict)
- Transactions that only write, never read (no read dependency)
- When all writes are independent (no shared read predicates)

**Cases where READ COMMITTED is fine:**

- Best-effort analytics queries
- Logging / append-only writes
- When stale reads are acceptable

---

## Retry Logic

**The application MUST catch SQLSTATE 40001 and retry.**

```
-- PostgreSQL raises this on serialization failure:
-- SQLSTATE: 40001
-- SQLSTR:   could not serialize access due to read/write dependencies

-- Also catch SQLSTATE 40P01 (deadlock detected) with same retry
```

Pseudocode retry pattern:

```
MAX_RETRIES = 5
INITIAL_BACKOFF_MS = 10

for attempt in range(MAX_RETRIES):
    try:
        with conn.transaction():
            conn.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
            # ... do work ...
            break  # success
    except SerializationFailure:  # SQLSTATE 40001
        if attempt == MAX_RETRIES - 1:
            raise
        sleep(INITIAL_BACKOFF_MS * (2 ** attempt) + random_jitter())
```

PL/pgSQL retry loop:

```
CREATE OR REPLACE FUNCTION book_seat_safe(p_show_id INT, p_seat TEXT, p_user_id INT)
RETURNS BOOLEAN AS $$
DECLARE
    v_attempt INT := 0;
BEGIN
    LOOP
        BEGIN
            SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

            IF (SELECT COUNT(*) FROM bookings
                WHERE show_id = p_show_id AND seat = p_seat) > 0 THEN
                RETURN FALSE; -- already booked
            END IF;

            INSERT INTO bookings(show_id, seat, user_id) VALUES (p_show_id, p_seat,
p_user_id);
            RETURN TRUE;

        EXCEPTION
            WHEN serialization_failure OR deadlock_detected THEN
                v_attempt := v_attempt + 1;
                IF v_attempt >= 5 THEN RAISE; END IF;
                PERFORM pg_sleep(0.01 * v_attempt);
        END;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

---

## SQL Examples

```
-- 1. Set serializable isolation
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- or
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- 2. Write skew scenario setup
CREATE TABLE doctors (
    name    TEXT PRIMARY KEY,
    on_call BOOLEAN NOT NULL DEFAULT true
);
INSERT INTO doctors VALUES ('Alice', true), ('Bob', true);

-- 3. Constraint that SSI will protect
-- (Business rule: at least 1 doctor on call — enforced by SSI, not a DB constraint)

-- Session 1:
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM doctors WHERE on_call = true; -- sees 2
```

```sql
UPDATE doctors SET on_call = false WHERE name = 'Alice';
COMMIT; -- may get serialization failure if session 2 ran concurrently

-- Session 2 (concurrent with session 1):
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM doctors WHERE on_call = true; -- sees 2
UPDATE doctors SET on_call = false WHERE name = 'Bob';
COMMIT; -- ERROR: could not serialize access (one of these will fail)

-- 4. Inventory reservation with SSI
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT available_qty FROM inventory WHERE product_id = 101;
-- If > 0, proceed with order
INSERT INTO order_items(order_id, product_id, qty) VALUES (1001, 101, 1);
UPDATE inventory SET available_qty = available_qty - 1 WHERE product_id = 101;
COMMIT; -- SSI ensures no oversell even with concurrent transactions

-- 5. Check serialization failure count
SELECT conflicts FROM pg_stat_database WHERE datname = current_database();

-- 6. SSI with read-only transaction (no overhead — fast path)
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE READ ONLY;
SELECT SUM(balance) FROM accounts; -- snapshot consistent, no predicate locks needed
COMMIT;

-- 7. Deferrable serializable (snapshot taken at COMMIT time, not BEGIN)
-- Only safe for read-only transactions, avoids false conflicts
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE READ ONLY DEFERRABLE;
-- May wait at BEGIN until a safe snapshot is available
SELECT * FROM large_report_table;
COMMIT;

-- 8. pg_stat_activity: find transactions in serializable mode
SELECT pid, application_name, state, query
FROM pg_stat_activity
WHERE backend_type = 'client backend'
  AND query LIKE '%SERIALIZABLE%';

-- 9. Monitor active predicate locks
SELECT l.pid, l.mode, a.query
FROM pg_locks l
JOIN pg_stat_activity a USING (pid)
WHERE l.mode = 'SIReadLock';

-- 10. Retry in application (Python-style comment pseudocode)
-- conn.autocommit = False
-- for attempt in range(5):
--     try:
--         cur.execute("BEGIN")
--         cur.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
--         ... business logic ...
--         conn.commit()
```

```
--         break
--     except psycopg2.errors.SerializationFailure:
--         conn.rollback()
--         time.sleep(0.01 * 2**attempt)

-- 11. Safe snapshot for reporting (avoids SSI overhead)
-- Use REPEATABLE READ for pure read-only analytics
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT region, SUM(revenue) FROM sales GROUP BY region;
COMMIT;

-- 12. Ticket booking with SSI
CREATE TABLE seats (
    show_id  INT,
    seat_no  TEXT,
    user_id  INT,
    PRIMARY KEY (show_id, seat_no)
);

BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- Check if seat available
SELECT COUNT(*) FROM seats WHERE show_id = 1 AND seat_no = 'A1';
-- 0 = available, proceed
INSERT INTO seats VALUES (1, 'A1', 42);
COMMIT; -- SSI prevents double-booking even without a second explicit check
```

---

## Common Mistakes

1. **Not implementing retry** — application catches 40001 but raises an error to the user instead of retrying. SSI is only useful if the application retries.
2. **Using SERIALIZABLE for everything** — high overhead. Use only when write skew is actually a risk. Most CRUD apps don't need it.
3. **Long transactions under SERIALIZABLE** — predicate locks held longer → more memory, more false conflicts. Keep SERIALIZABLE transactions short.
4. **Read-only SERIALIZABLE without DEFERRABLE** — `SERIALIZABLE READ ONLY` without `DEFERRABLE` still does predicate lock tracking. Add `DEFERRABLE` for true read-only reporting to get a safe snapshot with no overhead.
5. **Confusing 40001 and 40P01** — 40001 is serialization failure, 40P01 is deadlock. Both need retry, but different root causes. Both use the same retry pattern.

---

## Best Practices

1. Use **SERIALIZABLE READ ONLY DEFERRABLE** for long read-only transactions (reports, exports) — zero overhead.
2. Keep SERIALIZABLE read-write transactions **as short as possible** — less predicate lock memory, fewer conflicts.
3. Implement **exponential backoff with jitter** in retry logic — prevents retry thundering herd.

4. **Monitor conflict rate** via `pg_stat_database.conflicts` — if very high, reconsider design.
5. Consider whether a **database-level constraint or trigger** can enforce the invariant instead of relying on SSI.

---

## Performance Considerations

- SERIALIZABLE transactions use more memory (predicate locks) than REPEATABLE READ
- `max_pred_locks_per_transaction` (default 64) — tune if you have wide queries under SSI
- Read-only SERIALIZABLE DEFERRABLE: zero overhead (uses safe snapshot, no predicate locks)
- Abort rate under SSI is usually low for well-designed transactions (< 1%)
- If abort rate is high, application design likely has unnecessary read dependencies

---

## Interview Questions

**Q1: What is write skew and why doesn't REPEATABLE READ prevent it?** Write skew is when two concurrent transactions each read overlapping data and write to disjoint data, but together violate an application-level invariant. REPEATABLE READ prevents write-write conflicts (two transactions updating the same row) but not write skew because each transaction writes a DIFFERENT row — there is no direct conflict. The invariant violation is only visible when both writes are considered together.

**Q2: How does PostgreSQL's SSI detect serialization anomalies without preventing them?** SSI uses predicate locks (SIReadLock) to track what data each transaction reads. It then looks for rw-anti-dependency cycles: if T1's reads were written by T2, and T2's reads are written by T1, they cannot be serialized. When such a cycle is detected, PostgreSQL aborts one transaction with SQLSTATE 40001.

**Q3: What is the difference between SERIALIZABLE and REPEATABLE READ in PostgreSQL?** Both use MVCC snapshots. REPEATABLE READ only detects write-write conflicts (two transactions updating the same row). SERIALIZABLE also detects read-write anti-dependencies via predicate locks, preventing write skew and serialization anomalies. SERIALIZABLE guarantees that the result is equivalent to some serial execution of the transactions.

**Q4: What should an application do when it receives SQLSTATE 40001?** Roll back the transaction and retry it from scratch. The error message even includes a HINT saying "The transaction might succeed if retried." The retry should include exponential backoff with random jitter to avoid a retry storm. The number of retries should be bounded (e.g., 5 attempts).

**Q5: When would you use SERIALIZABLE READ ONLY DEFERRABLE?** For long-running read-only transactions like reports or data exports. The DEFERRABLE option allows PostgreSQL to wait until a "safe" snapshot is available — one that requires no predicate lock tracking. This eliminates the overhead of SSI for pure read workloads while still providing a fully consistent snapshot.

**Q6: What is a predicate lock in SSI?** A predicate lock (SIReadLock) is a lightweight lock that records which rows or ranges a transaction has READ, not for blocking purposes, but for dependency tracking. Unlike regular locks, predicate locks don't cause blocking — they only enable the cycle detection algorithm. They are released when the transaction commits.

**Q7: Can you force a specific transaction ordering under SERIALIZABLE?** Not directly — SSI decides which transaction to abort based on internal dependency tracking. If you need guaranteed ordering, use explicit application-level sequencing (e.g., process transactions one at a time through a queue) rather than relying on SSI ordering.

**Q8: What is the overhead of SERIALIZABLE compared to READ COMMITTED?** SERIALIZABLE adds predicate lock tracking (memory per read operation), cycle detection on commit (CPU), and potential transaction aborts requiring retries (latency). For typical OLTP workloads with short transactions and low conflict rates, overhead is usually 5-15%. For read-heavy workloads, use READ ONLY DEFERRABLE to eliminate SSI overhead entirely.

---

## Exercises

**Exercise 1:** Create the on-call doctors scenario and demonstrate SSI preventing write skew. Run two concurrent sessions and show that one gets a serialization failure.

**Solution:**

```
-- Setup
CREATE TABLE doctors (name TEXT PRIMARY KEY, on_call BOOLEAN DEFAULT true);
INSERT INTO doctors VALUES ('Alice', true), ('Bob', true);

-- Session 1
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM doctors WHERE on_call = true;  -- 2
UPDATE doctors SET on_call = false WHERE name = 'Alice';
-- Hold here, do not COMMIT yet

-- Session 2
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM doctors WHERE on_call = true;  -- still 2 (snapshot)
UPDATE doctors SET on_call = false WHERE name = 'Bob';
COMMIT; -- may succeed

-- Session 1
COMMIT; -- ERROR: could not serialize access (or session 2 got the error)
```

**Exercise 2:** Write a PL/pgSQL function that books a flight seat using SERIALIZABLE isolation with automatic retry (max 5 attempts).

**Solution:**

```
CREATE TABLE flight_seats (
    flight_id INT, seat TEXT, passenger_id INT,
    PRIMARY KEY (flight_id, seat)
);

CREATE OR REPLACE FUNCTION book_seat(
    p_flight INT, p_seat TEXT, p_passenger INT
) RETURNS BOOLEAN AS $$
DECLARE
    v_attempt INT := 0;
    v_exists  INT;
BEGIN
    LOOP
        BEGIN
            SET LOCAL TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
            SELECT COUNT(*) INTO v_exists
            FROM flight_seats WHERE flight_id = p_flight AND seat = p_seat;
            IF v_exists > 0 THEN RETURN FALSE; END IF;
            INSERT INTO flight_seats VALUES (p_flight, p_seat, p_passenger);
            RETURN TRUE;
        EXCEPTION
            WHEN serialization_failure OR deadlock_detected THEN
                v_attempt := v_attempt + 1;
                IF v_attempt >= 5 THEN RAISE; END IF;
                PERFORM pg_sleep(0.01 * v_attempt);
        END;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

**Exercise 3:** Explain which isolation level is appropriate for each scenario: (a) daily sales report, (b) bank transfer, (c) user profile update.

**Solution:**

- (a) READ ONLY REPEATABLE READ or SERIALIZABLE READ ONLY DEFERRABLE — consistent snapshot across all tables for the duration of the report.
- (b) SERIALIZABLE — prevents write skew where two transfers could both see sufficient balance and both proceed, overdrafting the account.
- (c) READ COMMITTED (default) — simple update, optimistic locking with a version column handles the rare concurrent edit case better than isolation-level overhead.

---

## Cross-references

- See 02_mvcc.md for snapshot internals that SSI builds upon
- See 03_isolation_levels.md for full isolation level comparison
- See 04_locks.md for regular locking (SSI predicate locks are separate)
- See 06_optimistic_vs_pessimistic.md for alternative to SSI for some use cases